MongoDB & Python – Partie 2 suite

Manipuler MongoDB à partir d'un programme Python en utilisant PyMongo pour :

- **créer des index**
- **effectuer des requêtes d'agrégation**
- **modifier les données (CRUD avancé)**

Accéder à MongoDB depuis Python (rappel)

PyMongo est la bibliothèque officielle qui permet à Python de communiquer avec MongoDB.

Connexion de base

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client.centreFormation
```

Explication :

- MongoClient : crée une connexion avec MongoDB
- centreFormation : base de données utilisée

Formateur : AARAB HAFID

Les Index MongoDB (via Python)

2.1 Définition

Un index est une structure spéciale qui permet à MongoDB de retrouver les documents plus rapidement,
exactement comme l'index d'un livre.

Sans index : MongoDB parcourt toute la collection
Avec index : MongoDB va directement à l'information

Voir les index existants

```
for nom, info in db.Stagiaires.index_information().items():
    print("Index :", nom, "→ champs :", info["key"])
```

Explication :

- index_information() : retourne tous les index
- chaque index possède un nom et un champ indexé

Créer un index simple

```
db.Stagiaires.create_index([("nom", 1)])
```

Explication :

- "nom" : champ indexé
- 1 : ordre croissant
- MongoDB accélère maintenant les recherches par nom

Supprimer un index

db.Stagiaires.drop_index("nom_1")

nom_1 est le nom interne généré par MongoDB

Pourquoi utiliser des index ?

- Requêtes plus rapides
- Très utiles sur les grandes collections
- Consomment un peu de mémoire
- Insertion légèrement plus lente
- À utiliser sur les champs souvent recherchés (nom, email, filière…)

Requêtes d'agrégation MongoDB

Définition

L'agrégation permet de calculer et analyser des données :

- **moyenne**
- **somme**
- **regroupement**

Équivalent SQL :

GROUP BY, AVG(), SUM()

MongoDB utilise une pipeline d'agrégation

Voir aussi sur le site référent :
https://www.w3schools.com/mongodb/mongodb_aggregations_intro.php

Principe de la pipeline

Une pipeline est une suite d'étapes :

Opérateur	Rôle
$match	filtrer
$group	regrouper
$avg	moyenne
$sum	somme
$sort	trier

Exemple : moyenne par filière

```
resultat = db.Stagiaires.aggregate([
  {
    "$group": {
      "_id": "$filiere.intitule",
```

```
            "moyenne": { "$avg": "$moy1A" }
        }
    }
])

for r in resultat:
    print(r["moyenne"], "moyenne de la filière", r["_id"])
```

Explication :

- $group : regroupe les documents
- _id : champ de regroupement
- $avg : calcule la moyenne
- resultat est un curseur, il faut le parcourir avec for

Requêtes de modification avec PyMongo

Insertion

```
db.Stagiaires.insert_one({"nom": "Ali", "age": 20})
db.Stagiaires.insert_many([
    {"nom": "Sara", "age": 21},
    {"nom": "Youssef", "age": 22}
])
```

Modification

```
db.Stagiaires.update_one(
    {"nom": "Ali"},
    {"$set": {"age": 21}}
)
db.Stagiaires.update_many(
    {"niveau": "1A"},
    {"$set": {"niveau": "2A"}}
)
```

Suppression

```
db.Stagiaires.delete_one({"nom": "Test"})
db.Stagiaires.delete_many({"niveau": "Ancien"})
```

En Résumé

- PyMongo permet de manipuler MongoDB depuis Python
- Les index améliorent les performances
- L'agrégation permet les calculs
- update / delete modifient les données

Sécurité MongoDB

Sécuriser une base MongoDB en maîtrisant :

- **Import / Export**
- **Authentification**
- **Rôles et droits**

Import / Export des données

Définition

Export : MongoDB → fichier
Import : fichier → MongoDB

Outils utilisés :

- **mongoexport**
- **mongoimport**

Formateur : AARAB HAFID

Export avec mongoexport

mongoexport -d centreFormation -c Stagiaires -o **C:/data/stagiaires.json**

Explication :

- **-d : base**
- **-c : collection**
- **-o : fichier de sortie**

Export CSV

mongoexport -d Examples -c Stagiaires -f _id,nom,prenom,moy1A --csv -o C:/data/Stagiaires.csv

Import avec mongoimport

mongoimport -d centreFormation -c Stagiaires C:/data/stagiaires.json

Authentification MongoDB

Définition

L'authentification = se connecter avec login + mot de passe

Création d'un utilisateur

```
use Examples
db.createUser({
  user: "UserAdmin",
  pwd: "password",
  roles: [
    { role: "userAdmin", db: "Examples" }
  ]
})
```

Activer la sécurité

Dans mongod.cfg :

```
security:
  authorization: enabled
```

Redémarrer MongoDB

Authentification

```
db.auth("UserAdmin", "password")
```

Rôles et droits MongoDB

Définition

Un rôle définit ce que l'utilisateur peut faire

Rôles principaux

Rôle	Description
read	lecture seule
readWrite	lecture + écriture
dbAdmin	administration
userAdmin	gestion utilisateurs
root	tous les droits

Afficher un utilisateur

db.getUser("UserAdmin")

Supprimer un utilisateur

db.dropUser("UserAdmin")